



Trinity College Dublin
Coláiste na Tríonóide, Baile Átha Cliath
The University of Dublin

FINAL YEAR PROJECT REPORT

TRINITY COLLEGE DUBLIN

THE UNIVERSITY OF DUBLIN

SCHOOL OF COMPUTER SCIENCE AND STATISTICS

Testing Encrypted ClientHello (ECH) in Apache

Author:

Alban Hideki Kataoka
22338084

Supervisor:

Dr Stephen Farrell

Submitted
April 8, 2026

A Final Year Project submitted in partial fulfillment of the degree of
BA Mod. Computer Science

Declaration of Work

I hereby declare that this dissertation is entirely my own work and that it has not been submitted as an exercise for a degree at this or any other university.

I have read and I understand the plagiarism provisions in the General Regulations of the University Calendar for the current year, found at <http://www.tcd.ie/calendar>.

I have completed the Online Tutorial on avoiding plagiarism 'Ready Steady Write', located at <http://tcd-ie.libguides.com/plagiarism/ready-steady-write>.

I consent / do not consent to the examiner retaining a copy of the thesis beyond the examining period, should they so wish (EU GDPR May 2018).

I agree that this thesis will not be publicly available, but will be available to TCD staff and students in the University's open access institutional repository on the Trinity domain only, subject to Irish Copyright Legislation and Trinity College Library conditions of use and acknowledgement. **Please consult with your supervisor on this last item before agreeing, and delete if you do not consent.**

Signed: _____

Date: _____

Abstract

Encrypted Client Hello (ECH) is a proposed extension to the Transport Layer Security (TLS) protocol that encrypts information currently leaked during connection. While ECH is beginning to see implementation and adoption across the Internet, particularly in modern web browsers, there remains a significant gap in standardised server-side verification tools. This project addresses this gap through the development of the first automated testing suite designed to validate ECH behaviour within the Apache HTTP Server.

Acknowledgements

Place acknowledgement here.

Contents

1	Introduction	1
1.1	Background	1
1.1.1	Encrypted Client Hello (ECH)	1
1.1.2	The Apache HTTP Server	1
1.2	Motivation	1
1.3	Aims and Objectives	2
1.4	Dissertation Outline	3
2	State of the Art	4
2.1	The Evolution of TLS Privacy	4
2.1.1	TLS 1.2: Plaintext Problems	4
2.1.2	TLS 1.3: The Privacy Gap	4
2.1.3	Encrypted SNI	4
2.1.4	Encrypted ClientHello (ECH)	5
2.1.5	The Verifiability Challenge	5
2.2	The ECH Operational Landscape and Dependencies	5
2.2.1	Hybrid Public Key Encryption (HPKE)	5
2.2.2	DNS HTTPS Resource Records	6
2.2.3	GREASE and Anti-Ossification Mechanisms	6
2.2.4	The Fallback Mechanism and Retry-Configs	6
2.3	Existing Implementations and Upstream Support	7
2.3.1	The Road to OpenSSL 4.0 and RFC 9849	7
2.3.2	NGINX vs. Apache	7
2.3.3	Justification for an Automated Test Suite	8
2.4	Existing Verification Methods	8
2.4.1	Browser-Based Diagnostics	8
2.4.2	OpenSSL s_client	8
2.4.3	Packet Analysis and Protocol Observation	8
2.4.4	Cloud-Based Verification Services	8
2.4.5	BoringSSL BoGo	9
2.4.6	TLS-Attacker and Protocol Fuzzing	9
2.4.7	testssl.sh and SSLyze	9
2.4.8	The Role of Browser Automation in Protocol Validation	9
2.4.9	Integrated Testing in the NGINX Ecosystem	10
2.4.10	The "Underlying Library" Dependency	10
2.4.11	Community-Driven Interoperability Verification	10
2.5	The Testing Gap in Apache httpd	10
2.5.1	Absence of Integrated Regression Tests	10
2.5.2	Cryptographic Implementation	11
2.5.3	The Observability and Debugging Gap	11
2.5.4	Configuration Complexity	11
2.5.5	Integration with Modern CI/CD Workflows	11
2.6	Summary	12

3	System Design	13
3.1	System Design Goals, Requirements and Constraints	13
3.1.1	Design Goals	13
3.1.2	System Requirements	13
3.1.3	Constraints from Existing ECH Architecture and the Testing Environment	15
3.2	Design Evolution and Rationale	16
3.2.1	Initial Development	16
3.2.2	Design Tradeoffs and Decisions	16
3.2.3	Shift to Truth Verification	17
3.3	Architecture Overview	17
3.3.1	Operational Lifecycle of a Test Case	18
3.3.2	The ECH Handshake Logic (Logical Design)	18
3.3.3	Verification Methodology	19
3.3.4	The Logical Trust and DNS Model	19
3.3.5	Negative Testing and Failure Detection	20
4	Implementation	21
4.1	Build Process and Experimental Setup	21
4.1.1	WSL and Development Environment Setup	21
4.1.2	Building the ECH-Enabled Cryptographic Library	21
4.1.3	Integrating and Compiling the Apache ECH Fork	22
4.1.4	Static Verification of ECH Support	22
4.1.5	Issues Encountered So Far	22
	Bibliography	25
A	First Appendix	26

List of Figures

1.1 Predicted Timeline of project as of 22nd of December, 2025	2
--	---

Chapter 1

Introduction

Internet privacy has continued evolving and has now moved toward protecting the identity of the destinations visited, as well as the data exchanged. This project aims to examine the implementation and verification of Encrypted ClientHello (ECH) within the Apache HTTP Server. This chapter provides context to the existing metadata leakage in TLS, the motivation for automated verification project, and the specific objectives of the project.

1.1 Background

1.1.1 Encrypted Client Hello (ECH)

The Transport Layer Security (TLS) protocol is the foundation of secure communication across the modern internet. It helps keep data private and secure by allowing the encryption of communication between a client and a server. However, when a TLS connection first starts, the client sends a message called the ClientHello to begin the handshake. In this message is a field called the Server Name Indication (SNI). This contains the hostname which the client wants to connect to.

This SNI is sent in plaintext, allowing anyone monitoring the network to see which website a user is visiting, even though the rest of the communication is encrypted. This can leak information about the user's activity and it can be used for tracking.

Encrypted ClientHello (ECH), standardised in March 2026 as RFC 9849, is an important enhancement to TLS 1.3 which fixes these metadata leaks. By encrypting the entire `ClientHello` message, ECH makes sure that sensitive extensions like the SNI are accessible only to the intended destination server. It hides the SNI and other sensitive parts of the handshake by encrypting the ClientHello message. Only the intended server has the right key to decrypt the message. This can prevent network observers from knowing which site a client is trying to access, improving privacy and security. Because ECH has recently transitioned to a finalised RFC, the industry is currently in a very underdeveloped state and the protocol exists in code but lacks standardised testing tooling.

1.1.2 The Apache HTTP Server

The Apache HTTP Server is one of the most widely used and long-standing web servers in the world. It is an open-source project maintained by the Apache Software Foundation and is known for its reliability, flexibility, and modular design which allows for the dynamic integration of security features through modules such as `mod_ssl`. Apache supports a broad range of features, including virtual hosting, authentication and SSL/TLS encryption.

As it is open-source and has a large user base, Apache is frequently used as a reference platform for testing new web technologies and standards. Its modular architecture allows developers to extend functionality easily, making it a suitable candidate for experimenting with Encrypted ClientHello (ECH) support and testing its impact on real-world server configurations.

1.2 Motivation

The motivation for this project is rooted in the observability paradox (as technology like ECH advances and connections become more secure, it becomes harder to verify their inner workings)

and the high technical barrier to verifying privacy-enhancing technologies. As Encrypted Client Hello (ECH) begins to see implementation in branches of the Apache HTTP Server and OpenSSL, the primary obstacle to its adoption is no longer just the code itself, but the verifiability of its deployment. There is a lack of standardised, automated testing frameworks that can be used to validate the functionality of these deployments. Specifically, in Apache, there are no existing ECH tests whatsoever.

Currently, a major obstacle to ECH adoption is not just the underlying C code, but the high technical requirements needed to ensure that the privacy features are functioning as intended rather than just silently falling back to insecure defaults. This project aims to help developers work around this obstacle with a comprehensive testing suite, providing the necessary tools to aid in the recent transition of ECH from an experimental draft to a ratified standard, it needs to be reliably verifiable in production deployment within the Apache web server. Developing a test suite is now a very important and urgent task as developers begin integrating the finalised RFC logic into production servers, they require a source of truth that can be used to verify that the privacy features are functionally sound.

1.3 Aims and Objectives

The central aim of this project is to develop and execute a comprehensive test suite that validates the integrity of ECH deployments on the Apache HTTP Server. This is achieved through the following objectives:

1. To establish a functional ECH-enabled testbed using experimental forks of OpenSSL and the Apache web server.
2. To design a series of test cases that distinguish between successful ECH handshakes and silent fallbacks to plaintext SNI.
3. To implement automated scripts capable of programmatically verifying the encryption status of the ClientHello.

In preparation to complete these objectives, I created the Gantt chart included in Fig. 1.3.1 to help plan my progress during the four months of work. While the final result turned out to be quite different from the initial plan and this timeline didn't end up being extremely representative of the timeline in actuality the overall structure of the timeline helped me envision a plan for the project, alongside a plan document that I created under the advice of Dr.Farrell when beginning the project, which I have included in the appendix.

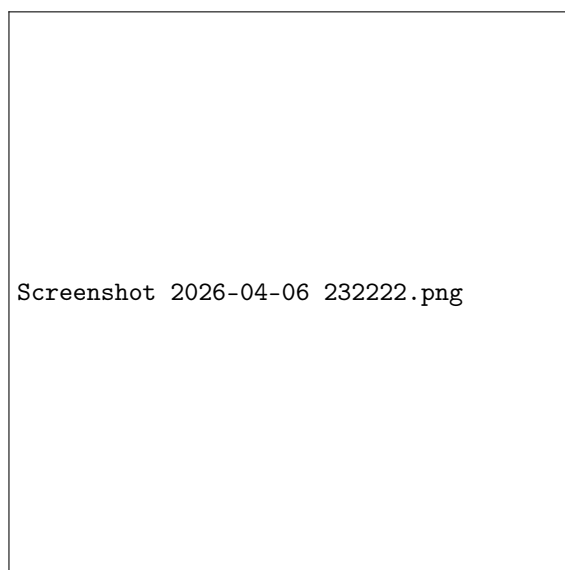


Figure 1.1: Predicted Timeline of project as of 22nd of December, 2025

1.4 Dissertation Outline

The remainder of this report is structured as follows:

- **Chapter 2: State of the Art** reviews the evolution of TLS, current experimental implementations of ECH and the existing testing methods.
- **Chapter 3: System Design** outlines the functional requirements of the testing suite and the architectural logic required to verify split mode handshakes as well as any design considerations that were made.
- **Chapter 4: Implementation** details the construction of the Python-based testing framework, the containerisation of the Apache testbed, and the integration of HPKE cryptographic functionality.
- **Chapter 5: Evaluation** analyses the project’s ability to detect misconfigurations and measures the computational overhead of ECH.
- **Chapter 6: Conclusion** summarises the impact of the suite on future Apache development, the effect it has had on me as a developer and my closing thoughts.

Chapter 2

State of the Art

This chapter explores the current state of Encrypted ClientHello (ECH) development and the existing methods for protocol verification, specifically dealing with the testing of technologies that enforce stricter privacy. By analysing the current and past IETF drafts, as well as the existing cryptographic tooling, this section identifies the testing gap that this project aims to address.

2.1 The Evolution of TLS Privacy

Transport Layer Security (TLS) is the cryptographic protocol used to provide secure communication over a computer network. The goal of the TLS protocol has been to provide a secure channel ensuring the confidentiality, integrity, and authenticity of application data. However, as time has gone on and the technology has evolved, the focus of the security community shifted toward the protection of connection metadata - the structural information required to route and manage the session—exposed in plaintext.

2.1.1 TLS 1.2: Plaintext Problems

Before TLS 1.3, a significant portion of the handshake occurred in the clear [1]. While the subsequent application data was encrypted, the initial exchange revealed the server’s digital certificate and the **Server Name Indication (SNI)** extension. The SNI was introduced in RFC 6066 to allow a single IP address to host multiple domains (Virtual Hosting). By sending the destination hostname in plaintext, this version of TLS 1.2 allowed pervasive surveillance and traffic filtering, as network observers could identify the user’s destination without needing to decrypt the actual payload.

2.1.2 TLS 1.3: The Privacy Gap

TLS 1.3 (RFC 8446) in 2018, brought a major leap in handshake privacy [2]. It introduced mandatory encryption for the server’s certificate, preventing observers from identifying the server identity via its public key credentials. Despite this, the SNI remained in the **ClientHello** message as plaintext. This was required for functionality as the server needs the SNI to select the correct certificate and configuration before encryption can be established. This created a privacy gap where the destination remained the only unencrypted piece of sensitive metadata.

2.1.3 Encrypted SNI

The first attempt to close this gap was **Encrypted SNI (ESNI)** [3]. This experimental extension allowed the client to encrypt only the SNI field using a key retrieved from a DNS record. However, ESNI was found to have its problems. While it hid the hostname, it left other extensions like the **Application-Layer Protocol Negotiation (ALPN)** exposed. An observer could still learn what kind of traffic it was by analysing these remaining plaintext fields. Furthermore, ESNI provided no mechanism for forward secrecy of the metadata, meaning a compromised DNS key could reveal past destinations.

2.1.4 Encrypted ClientHello (ECH)

Encrypted ClientHello (ECH), ratified as **RFC 9849** in March 2026, represents the standardised pinnacle of handshake privacy and addresses these problems by encrypting the entire ClientHello message [4]. It achieves this through a Split Mode architecture.

- **Inner ClientHello:** Contains the true, sensitive parameters of the intended session, including the private SNI and Application-Layer Protocol Negotiation (ALPN) tokens.
- **Outer ClientHello:** Acts as a non-sensitive placeholder message containing a public SNI that points to a common client-facing server or gateway.

With this architecture, a network observer only sees the connection to the public-facing gateway, while the true destination remains cryptographically hidden inside the encrypted payload, which is accessible only to the private server with the corresponding HPKE (Hybrid Public Key Encryption) private key [5].

By protecting the entirety of the sensitive information in these handshakes, ECH has removed the privacy issues that the earlier versions suffered from. However, this approach introduces a significant increase in the complexity of its implementation, as well as a new verifiability challenge.

In March 2026, the IETF formalised the protocol as RFC 9849. This transitioned ECH from an Internet-Draft to a proposed standard and marks the transition of ECH from an experimental privacy enhancement to a core component of the modern TLS stack. This formalisation provides the stable architectural baseline specifically regarding the handling of `ClientHelloInner` and `ClientHelloOuter` upon which the test suite in this project is built.

2.1.5 The Verifiability Challenge

In the previous iterations, a misconfiguration or a failed handshake would result in a hard connection failure. To prevent ossification (where network middleboxes block unknown extensions and break the internet), ECH is designed to be backward compatible. If a server receives an ECH extension it cannot decrypt (due to an expired key or misconfiguration), it does not terminate the connection. This results in a **silent fallback** where the user connects to the web server, but their privacy is silently lost as the connection switches to a standard TLS 1.3 handshake resulting in the issues from previous versions. In research or working environments, this fallback feature makes manual verification of the protocols functionality difficult, as a working connection is not guaranteeing that the ECH extension has made the connection private.

2.2 The ECH Operational Landscape and Dependencies

The implementation and testing of Encrypted ClientHello (ECH) within Apache's `mod_ssl` requires a deep integration with several external protocols and specific handling of TLS state transitions. This section outlines the cryptographic dependencies, discovery mechanisms and robustness features that form the state of the art for ECH today.

2.2.1 Hybrid Public Key Encryption (HPKE)

ECH leverages the **Hybrid Public Key Encryption (HPKE)** framework, as described in **RFC 9180** [5]. Unlike traditional public key schemes, HPKE provides a structured approach to combining asymmetric and symmetric primitives to ensure efficient, forward-secure encryption of the `ClientHelloInner`.

In the context of an ECH test, understanding the HPKE setup phase is important. This process involves:

- **Key Encapsulation Mechanism (KEM):** The client uses the server's public key (pkR) from the ECH configuration to generate an ephemeral key pair and a shared secret.
- **Key Derivation Function (KDF):** This secret is hashed with the context of the handshake to produce symmetric keys.
- **Authenticated Encryption (AEAD):** The resulting keys encrypt the sensitive metadata.

For this project’s test suite, the ability to manually construct these HPKE contexts is essential. If the suite cannot correctly encapsulate a key using the server’s advertised pkR , the server will be unable to decrypt the inner client hello, triggering the silent fallback behavior discussed in Section 2.1.5.

2.2.2 DNS HTTPS Resource Records

A significant shift in the state of the art for TLS 1.3 is the transition of key distribution from the handshake itself to the Domain Name System (DNS). ECH relies on the HTTPS Resource Record (RR Type 65), defined in RFC 9460 [6]

This record contains the echconfig payload, which includes the version, maximum name length, and the public key required for the HPKE operations. From a testing perspective, this introduces a new area possible failures:

- **Configuration Mismatch:** If the Apache server updates its private key but the DNS record remains cached with the old public key, ECH will fail.
- **Availability:** If the DNS resolver does not support Type 65 records, the client cannot retrieve the ECH configuration and will proceed with a standard, non-private handshake.

The testing suite developed in this project must therefore simulate a DNS aware client. It must be capable of parsing these records or, for local testing, manually injecting the echconfig parameters to verify how `mod_ssl` reacts when the provided key is intentionally malformed or expired.

2.2.3 GREASE and Anti-Ossification Mechanisms

A big component of the modern TLS ecosystem is **GREASE (Generate Random Extensions And Sustain Extensibility)**, as defined in **RFC 8701** [7]. The primary goal of GREASE is to prevent protocol ossification, where middleboxes (such as firewalls or load balancers) begin to reject unknown extensions, effectively freezing the protocol in its current state.

In the context of ECH, clients may send a dummy ECH extension containing random data. The server implementation like `mod_ssl` in Apache, must demonstrate the following behaviors:

- The server must ignore the dummy ECH extension and proceed with a standard TLS 1.3 handshake.
- The server must not terminate the connection or return a `handshake_failure` alert when encountering these reserved GREASE values.

For the development of this test suite, GREASE provides an important negative test category. The suite must verify that Apache correctly identifies the difference between a malformed `ClientHelloInner` (which should trigger a fallback) and a GREASE extension (which should be ignored).

2.2.4 The Fallback Mechanism and Retry-Configs

As an expansion on the aforementioned silent fallback, even when ECH is correctly configured, failures are expected due to the asynchronous nature of DNS. If a client uses an outdated echconfig from a cached DNS record, the Apache server will be unable to decrypt the `ClientHelloInner` using its current private keys.

Rather than failing the connection, the ECH specification defines a **Retry-Config** mechanism. In this state, the server completes the handshake using the `ClientHelloOuter` (the public identity) but includes a new, encrypted `encrypted_client_hello` extension in its `EncryptedExtensions` message. This extension contains the server’s current valid ECH configuration.

This introduces a complex state-machine requirement for a test suite:

1. **Configuration Provisioning:** The suite must simulate a stale client to trigger the fallback.
2. **Verification of Authenticity:** The suite must verify that the **Retry-Config** provided by Apache is syntactically correct and contains the expected public keys.
3. **Handshake Completion:** The suite must ensure that a subsequent handshake attempt using the provided **Retry-Config** results in a successful ECH-accepted connection.

This feedback loop is arguably the most difficult part of the ECH protocol to implement correctly, making it a high priority for our automated testing in `mod_ssl`.

2.3 Existing Implementations and Upstream Support

The landscape of Encrypted ClientHello (ECH) has recently transitioned from experimental research forks to formal standardisation and integration into core cryptographic libraries. As ECH moves toward a baseline feature of the internet, the implementation difference between major web servers like NGINX and Apache httpd provides a primary justification for a dedicated, standardised test suite.

2.3.1 The Road to OpenSSL 4.0 and RFC 9849

A defining shift in the state of the art for ECH is its formalisation in **RFC 9849** [4]. Earlier implementations relied on various iterations of the IETF drafts; however, RFC 9849 now serves as the definitive specification for HPKE suite negotiation, outer SNI handling, and retry mechanisms.

Along with the formalisation, ECH support has been merged into the **upstream master branch of OpenSSL**, with full native support officially scheduled for the **OpenSSL 4.0** release [8]. This is a critical development for Apache httpd for several reasons:

- **Legacy vs. Mainline:** Previously, Apache users required the *DEfO* (*Developing ECH for OpenSSL*) forks [9] to enable ECH. The shift to OpenSSL 4.0 removes the need for experimental patches but introduces the risk of API instability during the transition.
- **Implementation Responsibility:** With ECH logic residing in the core library, the role of Apache’s `mod_ssl` shifts toward the management of ECH configurations and the correct interfacing with DNS HTTPS records.

This transition marks the end of the experimental phase and begins the integration phase for web servers. However, it is important to distinguish between the current industry roadmap and the development environment of this project:

- While the industry is moving toward OpenSSL 4.0 and **RFC 9849**, much of the existing documentation and stable builds for Apache httpd still rely on the OpenSSL 3.x series.
- At the time of this project’s development, **OpenSSL 3.x (specifically the DEfO-maintained feature branches)** was utilized as the primary cryptographic provider. This version was chosen to ensure a stable baseline for testing while the OpenSSL 4.0 **master** branch undergoes active, breaking changes.
- A key objective of this test suite is to provide a validation framework that works with future versions. By targeting the protocol behaviors defined in RFC 9849 rather than specific library APIs, the suite remains relevant as Apache transitions from OpenSSL 3.x patches to native OpenSSL 4.0 support.

2.3.2 NGINX vs. Apache

Despite the movement toward OpenSSL 4.0, a disparity exists in how web servers currently expose ECH functionality to administrators.

Table 2.1: Comparison of ECH Integration Status (as of 2026)

Feature	NGINX (v1.29+)	Apache httpd (Upstream)
Support Status	Native / Upstreamed	Targeting OpenSSL 4.0 Integration
Specification	RFC 9849	RFC 9849 (via OpenSSL Master)
Configuration	Standardised <code>ssl_ech</code> directives	Transitioning from DEfO patches to native OpenSSL 4 APIs
Maturity	Integrated regression testing	Gap: Requires unified validation suite

As shown in Table 2.1, NGINX has achieved a level of ECH maturity [10] that Apache is currently working toward. While NGINX provides integrated observability (e.g., `$ssl_ech_status` variables), Apache’s transition to the OpenSSL 4.0 codebase requires a new layer of validation to ensure that `mod_ssl` correctly handles the new library’s state machine.

2.3.3 Justification for an Automated Test Suite

The move to OpenSSL 4.0 master does not eliminate the need for testing, instead, making it more important. As Apache developers integrate the new upstream APIs, they require a suite that can verify RFC compliance without relying on manual browser based tests.

By targeting the behaviors specified by RFC 9849 like correct **Retry-Config** issuance and GREASE handling, the suite ensures that as Apache moves from experimental forks to stable OpenSSL 4.0 support, it maintains the interoperability expected of an important server like Apache.

2.4 Existing Verification Methods

Current methods for verifying ECH status are largely fragmented and restricted to client-side observation or manual log analysis.

2.4.1 Browser-Based Diagnostics

Major browsers like Firefox and Google Chrome [11] have implemented internal connection identity panels. These can indicate whether ECH was accepted or bypassed. While useful for tech-savvy users who can now avoid that silent loss of privacy, these tools are unsuitable for systematic server protocol verification. With this feature, we cannot perform negative testing (e.g. verifying the server correctly rejects an incorrect key) or integrate it into a Continuous Integration (CI) pipeline for server-side developers.

2.4.2 OpenSSL `s_client`

The most technical method that is currently being used is the use of experimental forks of command-line tools like `openssl s_client`. These tools allow for the manual injection of echconfig strings and the inspection of hex-encoded handshake transcripts. However, as of 2024, the use of these tools is completely manual and there is no existing standardised framework that can use these tools to perform functional regression, interoperability, and performance testing specifically for the Apache HTTP Server environment.

2.4.3 Packet Analysis and Protocol Observation

Network protocol analysers like **Wireshark** and the command line based **TShark**, have initial support for looking into ECH-enabled handshakes. These tools can identify the presence of an ECH extension within a ClientHello and can distinguish between the Outer and Encrypted Inner components.

However, from a verification standpoint, packet analysis is limited by its design. Without the server's private HPKE key, a network administrator cannot programmatically verify that the Inner ClientHello actually contains the intended SNI. While these tools confirm that an ECH handshake is occurring, they cannot confirm if the handshake was actually performed correctly. This gap in truth verification means that packet capture alone is an incomplete solution for automated testing suites.

2.4.4 Cloud-Based Verification Services

As the primary drivers of ECH adoption, Content Delivery Networks (CDNs) such as Cloudflare have developed web-based security check tools. These services function by attempting a connection to a specific test endpoint and reporting back the negotiated parameters.

While effective for checking global internet connectivity and DNS-to-Server synchronisation, these services are fundamentally incompatible with private or local development environments. For a developer working on an experimental build of the Apache HTTP Server, an external cloud-based verification tool cannot reach a local testbed behind a firewall. This highlights the need for a local, reproducible test suite that can operate within a closed network environment, independent of third-party cloud infrastructure.

2.4.5 BoringSSL BoGo

The most comprehensive automated testing currently available for ECH is in the BoringSSL BoGo suite [12]. BoGo is a Go based test runner used to test the TLS state machine by creating malformed payloads and illegal protocol transitions. BoGo can simulate highly specific protocol edge cases, such as malformed HPKE payloads, ClientHelloInner compression errors, and forced HelloRetryRequest (HRR) scenarios. However, BoGo is designed to test the TLS library in isolation, using a simpler environment that does not account for the Apache process model, `mod_ssl` hooks, or virtual host routing. While BoGo can prove the library’s ECH logic is correct and working, it cannot verify that the full Apache production environment.

2.4.6 TLS-Attacker and Protocol Fuzzing

While functional testing ensures that a protocol operates under the ideal conditions, security research often utilises Fuzzing which is a technique that injects malformed to identify vulnerabilities in edge cases.

TLS-Attacker [13] is a Java based framework designed for the modular assembly of TLS handshakes. Unlike a standard client, it allows for the arbitrary reordering of messages and the intentional corruption of individual handshake fields. Recent research versions have began integrating ECH support to test the robustness of HPKE decryption logic and the server’s handling of the ClientHelloInner.

Additionally, projects like Tamarin or DYVerify attempt to mathematically prove the security properties of ECH. These tools look for logical flaws in the split mode architecture that could lead to identity leaks.

While TLS-Attacker is excellent for finding crashes and buffer overflows, it is a highly specialised tool for security researchers. It requires very specific expertise to configure and does not provide a simplified report on the passing of the protocol for a web server developer. These tools don’t cover the area that sits between basic functional checks and advanced cryptographic fuzzing. The tests I aim to develop can be used by an Apache developer to verify day-to-day configuration integrity in their development.

2.4.7 testssl.sh and SSLyze

For production server testing, `testssl.sh` and `SSLyze` are the most used tools for identifying cryptographic misconfigurations. However, their utility in verifying Encrypted ClientHello is currently limited by their reliance on stable, upstream library releases.

`testssl.sh` [14] is a bash based wrapper that relies on a locally compiled openssl binary to perform its checks. While `testssl.sh` is highly effective at identifying protocol versions and cipher suites, it does not currently feature a dedicated module for ECH truth verification. Since it mainly acts as a parser for OpenSSL output, it can only detect ECH if the underlying binary is a specialized fork. Even then, it lacks the logic to automate "negative testing" for ECH, such as verifying that a server does not leak the Inner SNI upon a decryption failure.

`SSLyze` [15] is a Python based library and CLI tool, `SSLyze` utilises `nassl`, a custom OpenSSL wrapper. While `SSLyze` provides a highly structured JSON output suitable for CI/CD pipelines, its ECH capabilities only go as far as identifying the encrypted `_client_hello` extension in the server’s EncryptedExtensions message. It currently lacks the high level API needed to construct a custom ClientHelloInner, meaning it cannot simulate the complex split mode handshakes required for a comprehensive testing suite.

2.4.8 The Role of Browser Automation in Protocol Validation

While not yet a standardised methodology for ECH verification, browser automation frameworks such as **Selenium WebDriver** represent a missing link in the current ECH testing scene. As discussed in Section 2.2.4, the primary risk of ECH is the silent fallback where a connection remains functional despite a privacy failure.

Existing tools like `s_client` verify the handshake but do not verify the application’s response to that handshake. Selenium provides the capability to programmatically interact with the browser itself to ensure the inner SNI is the one being rendered. It also allows the verification of ECH enabled features not breaking standard web functionality (e.g., cookie handling or redirects).

The absence of Selenium-based ECH testing in the current state of the art represents a significant gap in End-to-End (E2E) validation, which this project aims to bridge by integrating a WebDriver based testing layer.

2.4.9 Integrated Testing in the NGINX Ecosystem

A significant benchmark for this project is the existing integration testing framework provided by NGINX. As of version 1.29.4 [10], NGINX includes automated test cases within its primary `nginx-tests` suite. These tests utilize the `Test::Nginx` framework to verify RFC 9849 compliance by simulating various client handshake scenarios, including GREASE and malformed Inner ClientHellos.

Furthermore, NGINX provides high observability through native variables such as `$ssl_ech_status`, which allows for validation of the ECH state machine in production environments.

In contrast, Apache httpd currently lacks a centralised, automated framework for ECH validation. Testing is often fragmented, relying on developer-specific scripts or manual inspection of `mod_ssl` debug logs. This project directly addresses this state of the art deficiency by developing a standalone, automated test suite that brings NGINX level validation to the Apache ecosystem.

2.4.10 The "Underlying Library" Dependency

A recurring theme in the state of the art is the dependency on the OpenSSL 3.x stable branch. Because the IETF ECH draft has undergone numerous iterations, mainstream libraries have been hesitant to merge ECH support into their master branches to avoid breaking backward compatibility.

Auditors (`testssl.sh`) wait for Libraries (OpenSSL) to support ECH.

Libraries wait for the IETF Draft to become an RFC.

Developers are left with no way to test their implementations in the interim.

This project bypasses this deadlock by utilising a custom Python-based testing environment that leverages modern cryptographic bindings (via `cryptography.io`) to provide the ECH functionality that these popular auditors currently lack.

2.4.11 Community-Driven Interoperability Verification

Currently, the most complete ECH verification is not found in a single software tool, but rather in **IETF Interop Events**. These are collaborative hackathon style sessions where developers from the OpenSSL, BoringSSL, Cloudflare, and Akamai teams attempt to connect their respective clients and servers.

While these events have been instrumental in refining RFC 9849, they represent a significant bottleneck for independent web server developers. Interop results are often documented in temporary spreadsheets or chat logs, providing no long term regression testing for projects like Apache httpd. These events also require the presence of multiple experts to manually debug hex dumps of handshakes, a process that is not feasible for a standard DevOps or Web Admin workflow. Many interop tests focus on cloud implementations, possibly overlooking the configuration specifics of a locally hosted Apache instance.

This project aims to package the logic used in these high-level interop events into an automated, reproducible suite that can be run by an Apache developer without requiring a large testing committee.

2.5 The Testing Gap in Apache httpd

Despite the existence of the aforementioned tools, there is a clear lack of a unified, automated testing framework specifically for the Apache HTTP Server's implementation of ECH.

2.5.1 Absence of Integrated Regression Tests

The Apache Httpd project relies heavily on the `perl` framework [16] for functional testing. While this framework covers standard TLS handshakes, certificate validation, and `mod_ssl` configurations, it currently has zero test cases for ECH specific logic. Without an automated testing suite, Apache

developers must rely on manual testing, which is difficult and is unlikely to catch edge cases such as:

- **Key Rotation Failures:** How the server behaves when the ECH keys in the DNS do not match the keys in `httpd.conf`.
- **Virtual Host Collision:** Ensuring the server routes the decrypted Inner SNI to the correct virtual host rather than the Outer SNI host.
- **Performance Testing:** Measuring the overhead introduced by the HPKE decryption process on every new connection.

2.5.2 Cryptographic Implementation

To test ECH, the testing client must be capable of performing Hybrid Public Key Encryption (HPKE) as defined in RFC 9180. The legacy Perl based infrastructure lacks the modern cryptographic bindings required to generate the keys necessary for an ECH handshake [17]. This functionality requires that we move toward a modern environment containing a large variety of libraries and third party support like **Python/Pytest**, which can use updated OpenSSL bindings to simulate client behaviors.

2.5.3 The Observability and Debugging Gap

One of the most significant hurdles in the current Apache ECH landscape is the lack of visibility into the handshakes. In a standard TLS 1.3 environment, `mod_ssl` provides environment variables (e.g., `SSL_PROTOCOL`, `SSL_CIPHER`) [18] that allow administrators to verify the connection state.

However, ECH introduces a hidden internal state: the transition from the **Outer** to the **Inner** handshake. Currently, there is a gap in how Apache exposes whether an ECH decryption was successful or if a silent fallback occurred.

- Current debug logs in `mod_ssl` may show a successful TLS 1.3 handshake, but they do not explicitly flag the ECH Acceptance status in a format that is parsable by automated tools.
- There is no way to verify that the environment variables (like `HTTP_HOST`) were populated from the *Inner ClientHello* rather than the *Outer* decoy.

Without a suite that can query both the network handshake and the resulting server-side environment—developers cannot definitively prove that privacy was maintained.

2.5.4 Configuration Complexity

Unlike standard TLS features that are toggled with simple directives, ECH requires a synchronized coordination between the filesystem (private keys), the configuration file (`httpd.conf`), and the DNS (HTTPS records).

The Apache `perl-framework` is designed for static configuration testing. It is unequipped to handle the logic of ECH, where the server must attempt to decrypt an incoming packet using multiple possible keys without initially knowing which one the client used.

- **Dynamic Key Handling:** A testing gap exists in verifying how Apache behaves when multiple ECH configurations (different KEM/KDF/AEAD IDs) are active simultaneously.
- **The "Public Name" Paradox:** Testing must ensure that if ECH fails, the server strictly limits the client's access to the "Public Name" defined in the ECH configuration, preventing an unauthorized jump into a private `VirtualHost`.

2.5.5 Integration with Modern CI/CD Workflows

Finally, the existing Apache test infrastructure is largely legacy-based. Modern cryptographic development requires integration with CI/CD tools that support containerisation. The current reliance on Perl based scripts makes it difficult to inject the GREASE required by RFC 8701 [?]. By moving to a Python-based framework using `pytest`, this project addresses this gap, allowing for a more modular approach where individual ECH protocol steps can be isolated, mocked, and verified in a reproducible environment.

2.6 Summary

As demonstrated in this chapter, while Encrypted ClientHello (ECH) provides a cryptographic solution to the long-standing SNI leak problem, its practical deployment is hampered by a significant observability and verification vacuum. As it is a very new feature and is currently facing widespread adoption, a testing suite in a large server like Apache will go a long way to easing its development and adoption.

The current state of the art in ECH testing is incomplete and requires more focus:

- Library-level tools (such as BoringSSL’s BoGo) are highly effective at testing the cryptographic functionality of ECH but fail to account for the complex processing logic of a production web server like Apache.
- Network-level tools (such as Wireshark) are limited by the protocol’s design, as they cannot inspect the encrypted inner handshake without access to private key material.
- **Manual tools** such as experimental forks of `s_client` are unsuitable for modern CI/CD workflows and cannot reliably detect the silent fallback mechanism.

Specifically within the Apache `httpd` ecosystem, there is a clear absence of a unified, automated framework capable of validating the integration between `mod_ssl` and the ECH enabled libraries. This gap is particularly critical as the industry transitions from experimental DEfO patches toward native OpenSSL 4.0 support.

The remainder of this dissertation addresses this gap identified in this chapter. The requirements identified in this chapter, specifically the need for HPKE-aware automation, negative testing for GREASE, and End-to-End browser validation, will be addressed in the following chapters, detailing the design and implementation of a specialised testing suite. This suite aims to provide Apache developers with a verification tool that ensures privacy is not just a theoretical protocol feature, but a functional, verified feature in the Apache environment.

Chapter 3

System Design

This chapter describes the architectural design of the ECH test suite, including any considerations made for the design, constraints on the design and other possible choices that could have been made. It outlines the requirements for the experimental testing suite and defines the methodology we use to programmatically detect protocol success, failure and fallback.

3.1 System Design Goals, Requirements and Constraints

The design of the ECH test suite is focused on connecting the low-level cryptographic library testing and high level application server configuration. This chapter outlines the primary goals, functional requirements, and the architectural framework required to verify Apache `mod_ssl` according to RFC 9849.

3.1.1 Design Goals

The following goals serve as a guide towards the development of the testing framework to make sure that it is rigorous and technically covers all of the technical aspects of ECH required to make development easier:

- **RFC 9849 Compliance:** The suite must have basic test cases that serve as a reference implementation for validating the servers behavior against the finalized IETF standard, specifically focusing on the transitions between the `ClientHelloOuter` and `ClientHelloInner`.
- **End-to-End (E2E) Truth Verification:** Beyond simple handshake completion, the suite must verify that the inner application data (e.g. specific `VirtualHost` content) is served only when successful ECH decryption is completed.
- **Negative Testing:** Ensuring the server fails safely is an important goal. This includes verifying that malformed HPKE payloads, expired keys, or mismatched `public_name` values do not leak sensitive SNI information.
- **Environment Agnosticism:** The suite must be able to test Apache across different library versions (e.g., OpenSSL 3.x with DEFO patches vs. native OpenSSL 4.0) without requiring changes to the core test logic. It should also be reproducible on other machines.
- **Automated Observability:** The design aims to programmatically extract hidden TLS states such as ECH acceptance status that are currently difficult to parse from standard Apache error logs.

3.1.2 System Requirements

To meet these goals, the system must satisfy a rigorous set of functional and technical requirements.

Cryptographic Requirements (HPKE)

The testing client must implement the Hybrid Public Key Encryption (HPKE) primitives defined in RFC 9180 [5]. The system must be capable of performing the following operations:

- **SetupBaseOut:** The client must generate an encapsulated key and an encryption context using the server's public key extracted from the `ECHConfig`.
- **Context Encryption:** The suite must be able to encrypt the `ClientHelloInner` to produce the ciphertext sent in the ECH extension.
- **Suite Negotiation:** Support for the mandatory-to-implement (MTI) cipher suites, specifically KEM: DHKEM(X25519, HKDF-SHA256).

Functional Requirements

- **Handshake Orchestration:** The suite must support the full TLS 1.3 handshake, including the handling of `HelloRetryRequest` (HRR) scenarios where the server requires a stateless cookie or a different key share.
- **Decoy Service Validation (Public Name):** The system must verify that if ECH decryption fails (due to a malformed extension or key mismatch), the server strictly restricts the client to the resources of the `public_name` `VirtualHost`. The suite must confirm that no sensitive headers or content from the private SNI are leaked in the cleartext fallback response.
- **Retry-Config Mechanism Support:** In compliance with RFC 9849, the suite must be capable of parsing the `retry_configs` provided by Apache in the `EncryptedExtensions` message. It must verify the validity of the syntax of these configurations and demonstrate that a retry handshake using the updated keys results in a successful ECH accepted connection.
- **GREASE and Anti-Ossification Testing:** The suite must provide the ability to inject GREASE ECH extensions. It must verify that Apache ignores these extensions and proceeds with a standard TLS 1.3 handshake without terminating the session, as described in RFC 8701 [7].
- **Padding and Serialisation:** To prevent traffic analysis, ECH requires the `ClientHelloInner` to be padded to a certain length. The suite must verify that Apache correctly handles various padding lengths and does not reject handshakes where the inner message includes the maximum length of padding.
- **DNS Simulation and Record Parsing:** The system must provide a mechanism to provide the `ECHConfig` payload to the client, simulating the retrieval of **HTTPS RR (Type 65)** records [6]. This includes testing the client's ability to select the correct configuration when multiple records are advertised by the server.
- **Browser-Engine Consistency (Selenium):** The suite must utilise **Selenium WebDriver** (configured with ECH capable versions of Firefox or Chrome) to ensure that the application layer correctly identifies the `ClientHelloInner` SNI. This requirement is vital for verifying that session handling mechanisms, such as Cookies and CSRF tokens, are correctly interacting with the private `VirtualHost`.
- **Silent Fallback Detection:** A primary goal of the Selenium integration is to detect silent failures. The suite must programmatically inspect the browser's security state to confirm that ECH was successfully performed. It must verify that a page rendered for the private SNI was not actually served over an outer connection, which would indicate a privacy leak.
- **Handshake Latency Measurement:** The suite must provide a timing measurement of the TLS handshake to give a number on the overhead of HPKE decryption. It must compare the latency of an ECH accepted handshake against a standard TLS 1.3 handshake to establish a performance baseline for `mod_ssl`.

Technical Stack Requirements

The implementation utilises a modern, modular stack to ensure compatibility with modern CI/CD workflows:

- **Language:** Python 3.10+ for its extensive support for asynchronous I/O and cryptographic scripting.
- **Cryptographic Provider:** The `cryptography.io` library to handle low level HPKE primitives and TLS extension construction.
- **Test Runner:** `pytest` for modular test execution, parameterisation of cipher suites, and XUnit-style reporting.
- **Containerisation:** Docker based deployment to provide a more easily reproducible environment containing the specific Apache `httpd` builds and OpenSSL versions being tested.

Environmental and Library Requirements

A specific cryptographic toolchain and trust model are needed to ensure the suite can operate in local development environments.

- **OpenSSL Branch Synchronisation:** The system design assumes a dependency on specific OpenSSL versions. The suite must detect the linked OpenSSL version of the target Apache binary and adjust its expectations for `Retry-Config` behavior accordingly.
- **Apache `mod_ssl` Integration:** The suite must interact with specific `mod_ssl` directives that bridge the web server to the underlying library.

3.1.3 Constraints from Existing ECH Architecture and the Testing Environment

The design and implementation of the test suite is constrained by several technical and environmental issues/obstacles that influenced the architectural choices of the project.

- **Library Dependency and API Stability:** Since ECH support has been recently standardised in **RFC 9849**, the suite is dependent on specific, non-stable branches of OpenSSL (e.g. DEFO-patched 3.x) to enable ECH with the current upstream build of Apache. The suite must operate within the constraints of these more experimental APIs, which may lack certain hooks available in more mature TLS features.
- **Network Isolation and DNS Limitations:** Because the suite is designed for local development and CI/CD environments, it cannot use public DNS infrastructure. This causes a constraint on the project where **HTTPS RR (Type 65)** records must be manually injected or mocked, meaning that the suite does not test the actual network layer latency of a real DNS lookup.
- **Hardware-Specific Performance Timing:** Handshake latency measurements can possibly be affected by the hardware of the test runner. This can cause performance results to be slightly inaccurate and are mainly intended for relative comparison between ECH and non-ECH handshakes rather than as absolute benchmarks for production deployments.
- **Root Access and Containerisation:** The suite needs to be able to modify system level configurations and certificates within the target environment. This constrains the deployment to environments that support Docker or have elevated privileges, as a non-privileged user would be unable to reconfigure Apache's `mod_ssl` or the system trust store.
- **Use of Local Root CA:** The reliance on a local Root CA causes a difference in the test suites performance of an ECH handshake and a live ECH environment. In a production scenario, the client must validate a chain of trust involving multiple CAs and cross-signed roots. By utilising a single local CA, the suite bypasses potential issues related to certificate chain length, path discovery, and the asynchronous nature of Online Certificate Status Protocol (OCSP) checks. While this creates a stable testing baseline, it limits the suite's ability

to identify edge cases where ECH decryption succeeds but the resulting certificate chain is considered untrusted or malformed by the client.

- In reality, ECH security works with the DNS (for the `ECHConfig`), the CA (for the certificate), and the Server. By acting as the CA locally, the test suite creates a synthetic truth. If the suite generates a certificate that matches a manually injected `ECHConfig`, it proves the protocol logic works, but it does not test the real-world risk where a DNS record might be updated faster or slower than the certificate deployment at the CA level.

3.2 Design Evolution and Rationale

The current architecture of the ECH test suite is the result of an iterative design process. This section describes the transition from a manual, host-based testing environment to a containerised, more modern, automated framework, detailing the technical tradeoffs that informed these choices.

3.2.1 Initial Development

Initial development started with a manual integration of an ECH enabled Apache fork (using `mod_ssl` patches) and a custom OpenSSL feature branch. This phase revealed significant circular dependencies in the ECH ecosystem:

- It was discovered that even if an OpenSSL binary supports ECH commands, Apache cannot enable ECH features unless the OpenSSL headers explicitly expose the `SSL_OP_ECH_GREASE` and `HAVE_OPENSSL_ECH` macros at compile-time.
- Early iterations relied on the `SSLECHKeyDir` and `SSLECHConfigFile` directives. Mismatches between the OpenSSL version and the expected Apache API often led to silent failures where ECH code was compiled but remained inactive.

3.2.2 Design Tradeoffs and Decisions

The shift to the current system design was driven by three primary technical trade-offs:

C vs. Python for the Testing Logic

While the server-side code (Apache/OpenSSL) is natively written in **C**, I chose to use **Python** for the test suite.

- **Tradeoff:** C offers high performance cryptographic primitives, but Python provides easier use for developers. For the use of writing tests, the benefits of being faster and more powerful from C aren't important as we only need to run a few tests. Python allows the writing, breaking and changing of tests quickly without worrying about crashing.
- **Rationale:** The goal of the suite is not to perform high-speed encryption, but to perform *state-machine validation*. Python's `cryptography.io` allows us to manually manipulate TLS extensions more easily than a compiled C client.

Local Installation vs. Containerisation through Docker

Earlier attempts used local builds in the `apache-ech-install` directory.

- **Tradeoff:** Local builds are faster to compile but suffer from problems where system wide libraries interfere with the custom OpenSSL build.
- **Rationale:** I moved to **Docker** to ensure a consistent Apache environment. This allows me to strictly control the `LD_LIBRARY_PATH` and ensure Apache is linking against the exact ECH patched OpenSSL headers required for the `HAVE_OPENSSL_ECH` flag to trigger.

Live DNS vs. Config Injection

I considered implementing a full DNS stack to serve HTTPS Resource Records for the sake of fully authentic tests.

- **Tradeoff:** A live DNS stack is more representative of the real world but introduces latency and caching issues that make automated testing non-deterministic, and puts less focus on the actual server logic.
- **Rationale:** I chose to use direct config injection. By passing the ECHConfig directly to the client, we can isolate protocol failures from DNS resolution failures, which I found was extremely important for a regression testing suite.

Transition to a Local Trust Anchor (Root CA)

Another important decision in the development of the test suite was the implementation of a full-scale local Root Certificate Authority (CA) rather than utilising certificate verification overrides.

- **Initial Approach:** Early development utilised insecure modes to ignore certificate mismatches. While this allowed for rapid testing of the HPKE decryption, it failed to simulate the post-decryption behavior of a real browser.
- **Rationale:** In ECH, the certificate presented by Apache is tied to the Inner SNI. To fully verify the the handshake, the client must be able to validate the certificate chain. By creating a local Root CA, the test suite is ensuring that it behaves exactly like a production client, requiring Apache to present a valid, cryptographically signed identity for the hidden site.
- **Trade-off:** Automated CA management significantly increased the complexity of the Python orchestration logic. However, this was a necessary change to prevent false positives where a handshake succeeds at the TLS layer but fails at the application layer due to a chain-of-trust error.

3.2.3 Shift to Truth Verification

I discovered that Apache could silently disable ECH leading to a fundamental change in the testing methodology. Passive verification was originally the plan (checking if the server starts), but this had to be changed to active truth verification.

By this, I mean that the suite uses the server's application response as the source of truth. By configuring a secret token on the private VirtualHost that is inaccessible via the public _name, we can definitively prove that ECH decryption and routing were successful. This eliminates the risk of false positives where a standard TLS 1.3 handshake is mistaken for a successful ECH handshake.

3.3 Architecture Overview

Given the goals and constraints detailed in the previous sections, the architecture of the ECH test suite is designed as a three-tier system. This structure makes it so that the complex cryptographic operations of the client are separated from the server side configuration, allowing for precise control over the TLS handshake configurations. This separation of concerns also allows for easier use of the cryptographic and network configurations.

Three-Tier Environment

The system is divided into the following logical layers:

1. **Host Layer:** Developed in Python using the `pytest` framework, this layer acts as the brain of the suite. It is responsible for generating the Root CA, signing certificates, and dynamically spawning the target environment.
2. **Bridge Layer** A dedicated Docker bridge network provides a fresh environment for the experiments. By utilising a private subnet, the suite can simulate DNS behaviors and map hostnames (e.g., `public.example.com` and `secret.example.com`) to the container's internal IP without touching the host machine's network.

3. **Service Layer:** This layer houses the specific build of Apache linked against the ECH enabled OpenSSL library. It responds to handshakes based on the configuration injected by the host layer.

Component Decomposition

To achieve the goal of truth verification, the system is broken down into several specialised components:

ECH-Aware Mock Client: Unlike a standard browser, which is designed to hide protocol failures from the user, the mock client is built using `cryptography.io` to provide raw access to the TLS record layer. This allows the suite to create incorrect `EncryptedClientHello` extensions, vary padding lengths, and handle the `Retry-Config` requests programmatically.

The Dynamic Configuration Engine: This component generates the `httpd.conf` and SSL directives for each test case. It ensures that the `SSLECHKeyDir` points to the correct key material and that the VirtualHost routing is aligned with the `public_name` defined in the `ECHConfig`.

The Verification Logging Auditor: Once a handshake is completed, this engine performs a two part check. First, it inspects the HTTP response body for the presence of the unique secret token that has been placed on the private site. Second, it attaches to the Docker container's log stream to verify that `mod_ssl` recorded the ECH acceptance or fallback event with the expected diagnostic flags as defined in the file.

Data Flow and Key Management:

The flow of data through the architecture follows a strict path to ensure a valid setup for the handshakes between the client and server. Before a test begins, the host layer will generate an ECH Key Pair using a shell script which then encodes the public portion into an `ECHConfig` structure. This structure is then injected into the Apache configuration (as a file) and the mock client, ensuring that both ends of the handshake are using identical cryptographic parameters.

3.3.1 Operational Lifecycle of a Test Case

For the sake of total isolation and reproducibility, each test case follows a four stage execution:

1. **Provisioning Phase:** The Host Layer generates a unique HPKE key pair and a fresh set of X.509 certificates. It dynamically writes the `httpd.conf` and triggers the Docker API to spawn a clean container instance.
2. **Execution Phase:** The Mock Client constructs the dual layered `ClientHello`, performs the HPKE encryption, and initiates the TLS 1.3 handshake. Upon a successful encrypted session, it performs an HTTP GET request for the protected resource.
3. **Test Phase:** The suite retrieves the HTTP response and simultaneously scrapes the container's STDOUT log stream to capture `mod_ssl` trace events.
4. **Reset Phase:** The container is destroyed and all temporary cryptographic material is removed from the host. This clean slate then makes sure that no state persists between individual test scenarios.

3.3.2 The ECH Handshake Logic (Logical Design)

The core logic of the test suite follows the split-mode client architecture. Unlike a standard TLS client, the suite must maintain two different handshake states: the `ClientHelloInner`, which contains the sensitive SNI and extensions, and the `ClientHelloOuter`, which serves as the encrypted envelope which carries the inner.

Handshake Orchestration Sequence

The logical flow of a single test case is executed in the following stages:

1. The suite generates the `ClientHelloInner` (encoded as a handshake message). It calculates the necessary padding to ensure the inner message length doesn't leak the domain name size.

2. Using the ECHConfig retrieved from the configuration engine, the suite performs an HPKE SetupBaseOut operation. This creates the `encapsulated_key` and an encryption context.
3. The suite encrypts the `ClientHelloInner` using the context's seal function. The `ClientHelloOuter` is then created, with the resulting ciphertext placed in the `encrypted_client_hello` extension.
4. The client transmits the `ClientHelloOuter`. The tests then monitors the server's response for specific TLS alerts or the presence of a `HelloRetryRequest` (HRR). If an HRR is received, the suite calculates the transcript hash again and seals the inner handshake accordingly.

3.3.3 Verification Methodology

The suite defines a successful test not just by the completion of a cryptographic handshake, but by the verifiable isolation of application data. This is achieved through a multi layered assertion logic.

Source of Truth

To prevent false positives where a server might silently ignore ECH and fall back to a standard handshake, the test suite utilises a token based verification system.

- **The Private Token:** A unique string served only by the private `VirtualHost` (`secret.example.com`).
- **The Public Decoy:** A generic landing page served by the `public_name` (`gateway.example.com`).

Decision Matrix and Assertions

Each test case is evaluated against the following decision matrix to determine the final status:

Client State	Expected Response	Result
Valid ECH Config	Private Token	Success
Invalid Key	Public Decoy	Safe Fallback
Malformed Ciphertext	TLS Alert / Decoy	Negative Success

Table 3.1: ECH Test Decision Matrix

Log-Based Observability Audit

Finally, the suite performs a check of the Apache logs. Using the Docker API, the python scripts retrieve the last 50 lines of the `error_log`. It searches for specific `mod_ssl` trace symbols (e.g., `SSL_ECH_ACCEPTED` or `SSL_ECH_OUTER_SNI_MATCH`) to confirm that the server reached the same conclusion as the client.

3.3.4 The Logical Trust and DNS Model

To simulate a production level ECH environment, the suite must deal with how a client discovers ECH keys and how it verifies the server's identity once the secret SNI is decrypted.

Private Root CA and Chain of Trust

For ECH to be verified authentically, the client must perform a full X.509 validation of the certificate presented by the inner `VirtualHost`. To do this programatically, the host layer uses a **Private Root Certificate Authority (CA)**.

- **Design Logic:** The orchestrator generates a self-signed Root CA and uses it to sign unique leaf certificates for both the public and private hostnames.
- **Trust Injection:** The suite injects this Root CA into the client's trust store (e.g., the `certifi` store for Python or the `enterprise_roots` for Firefox). This ensures that the suite validates the ECH-protected identity using standard PKI logic.

DNS Abstraction via Configuration Injection

In a real-world scenario, ECH keys are retrieved via **HTTPS Resource Records (Type 65)** from a DNS server. To isolate the test suite from the complexities and caching latencies of DNS infrastructure, the suite directly injects the configurations.

- **Mechanism:** The ECHConfig (containing the public key and cipher suites) is generated by the host layer and passed directly to the mock client.
- **Rationale:** This ensures that a test failure is for sure caused by the TLS 1.3/ECH logic in Apache rather than a DNS resolution error or a delay caused by any networking issues. It provides a deterministic environment where the client is guaranteed to use the exact key material currently loaded into the server’s memory.

3.3.5 Negative Testing and Failure Detection

A system verifying the effectiveness of a server needs to ensure the server does not leak sensitive information when the ECH handshake is compromised. The suite observes certain failure points which trigger and detect these failures.

Protocol-Level Negative Testing

In certain test cases, the suite provides the client with a public key that does not match the server’s private key. The design expects Apache to detect the decryption failure and restrict the client’s access to the decoy site.

To test for anti-ossification, the suite injects unknown ECH extensions (GREASE). The design verifies that Apache ignores these values and proceeds with a standard handshake without terminating the connection. To do this, an EncryptedClientHello is constructed where the Outer SNI does not match the public name defined in the ECHConfig. This tests the server’s ability to obey the protocol’s routing requirements.

Server Robustness and Startup Validation

Beyond the TLS handshake, the suite aims to verify the server’s operational stability when ECH dependencies are missing or corrupted. These tests cases are important for assessing the whether the experimental `mod_ssl` fork is production ready or not: In these cases, the suite programmatically removes or renames ECH key files (`ECH_key.pem`) before a server restart. The design expects the server to terminate the process rather than starting in an insecure, non ECH state. By injecting incorrect syntax into the `httpd.conf` (e.g. corrupting the `SSLECHKeyDir` directive), the suite verifies that the Apache fork correctly implements syntax checking and provides diagnostic logs rather than attempting to start with an invalid configuration.

Chapter 4

Implementation

4.1 Build Process and Experimental Setup

This section documents all steps taken so far to build an Apache HTTP Server prototype with ECH (Encrypted Client Hello) support. This includes environment setup, compilation of dependencies, the integration of OpenSSL’s experimental ECH implementation, and all issues currently blocking progress.

4.1.1 WSL and Development Environment Setup

- Installed Windows Subsystem for Linux (WSL2) with Ubuntu.
- Configured directory structure (`~/projects`) for Apache, OpenSSL, and documentation.
- Fixed an issue where WSL directories were not appearing in the Windows filesystem (resolved by restarting WSL and ensuring the path under `wsl.localhost` was mounted).
- Installed required development tooling:
 - `build-essential`, `autoconf`, `automake`, `cmake`, `pkg-config`, `git`
 - Corrected a partially-applied `dpkg` transaction using: `sudo dpkg --configure -a`

4.1.2 Building the ECH-Enabled Cryptographic Library

Standard OpenSSL distributions do not yet support the Encrypted Client Hello (ECH) protocol. Therefore, the `sftcd` fork of OpenSSL, which serves as the primary reference implementation for the IETF draft, was utilized.

- **Repository:** Successfully cloned the `sftcd/openssl` repository and switched to the ECH feature branch:

```
git clone https://github.com/sftcd/openssl.git
cd openssl
git checkout feature/ech
```

- **Configuration:** The library was configured with a local prefix to prevent conflicts with the system’s native OpenSSL:

```
./config --prefix=$HOME/final_year/build/openssl \
--openssldir=$HOME/final_year/build/openssl
```

- **Compilation:**

```
make -j$(nproc)
make install_sw
```

- **Verification:** Confirmed ECH symbol presence in the library headers:

```
grep "SSL_OP_ECH_GREASE" $HOME/final_year/build/openssl/include/openssl/ssl.h
```

4.1.3 Integrating and Compiling the Apache ECH Fork

The primary challenge in this stage was ensuring the Apache `mod_ssl` module correctly detected and linked against the experimental TLS extensions.

1. **Pre-compilation Configuration:** Used `buildconf` to generate the necessary construction scripts.
2. **Linking and Feature Activation:** Since the `configure` script often fails to automatically detect experimental TLS extensions, the `HAVE_OPENSSL_ECH` flag was manually injected via `CPPFLAGS`. This processor activates the ECH code blocks within the source.

```
export PKG_CONFIG_PATH=$HOME/final_year/build/openssl/lib64/pkgconfig

./configure --prefix=$HOME/final_year/build/apache \
            --enable-ssl \
            --with-ssl=$HOME/final_year/build/openssl \
            --enable-mods-shared=all \
            CPPFLAGS="-I$HOME/final_year/build/openssl/include -DHAVE_OPENSSL_ECH" \
            LDFLAGS="-L$HOME/final_year/build/openssl/lib64 \
                    -Wl,-rpath,$HOME/final_year/build/openssl/lib64"
```

Build and Installation:

```
make -j$(nproc)
make install
```

4.1.4 Static Verification of ECH Support

To verify that the resulting binary contains the necessary ECH logic, the symbol table of the `mod_ssl.so` shared object was inspected.

```
nm ~/final_year/build/apache/modules/mod_ssl.so | grep -i "ECH"
```

The presence of symbols such as `ssl_callback_ECH` and `ssl_cmd_SSLECHKeyDir` confirms that the Apache module is ECH-aware and successfully linked against the experimental OpenSSL version tags (e.g., `@OPENSSL3.6.0`).

4.1.5 Issues Encountered So Far

1. ECH Key Generation Not Supported

The OpenSSL `ech` command exists, but only supports:

- `-out`
- `-in`
- `-select`
- `-public_name-suite`
All expected arguments from old drafts (`-gen`, `-pubout`, `-secretout`) are missing. Attempts such as:

- `openssl ech -gen ech.json`
`openssl ech -pubout ...`
`openssl hpke -keygen`

returned either:

Unknown option
OSSL_ECHSTORE_new_config error
Invalid command

Key conclusion: The OpenSSL `feature/ech` branch no longer includes a complete ECH key generation tool.

Bibliography

- [1] Tim Dierks and Eric Rescorla. The Transport Layer Security (TLS) Protocol Version 1.2. RFC 5246, RFC Editor, Fremont, CA, aug 2008. URL <https://www.rfc-editor.org/rfc/rfc5246>.
- [2] E. Rescorla. The transport layer security (tls) protocol version 1.3. RFC 8446, IETF, 2018. URL <https://www.rfc-editor.org/rfc/rfc8446.html>.
- [3] E. Rescorla, K. Oku, N. Sullivan, and C. Wood. Encrypting the server name indication in tls. Internet-Draft draft-ietf-tls-esni-00, IETF, July 2018. URL <https://datatracker.ietf.org/doc/html/draft-ietf-tls-esni-00>. Expired Internet-Draft.
- [4] C. A. Wood, N. Sullivan, E. Rescorla, and K. Oku. Tls encrypted client hello. RFC 9849, IETF, March 2026. URL <https://www.rfc-editor.org/rfc/rfc9849.html>.
- [5] R. Barnes, K. Bhargavan, I. Lippert, and C. Wood. Hybrid public key encryption. RFC 9180, IETF, 2022. URL <https://www.rfc-editor.org/rfc/rfc9180.html>.
- [6] B. Schwartz, M. Bishop, and E. Nygren. Service binding and parameter specification via the dns (dns svcb and https resource records). RFC 9460, IETF, 2023. URL <https://www.rfc-editor.org/rfc/rfc9460.html>.
- [7] D Benjamin. Applying Generate Random Extensions And Sustain Extensibility (GREASE) to TLS Extensibility. RFC 8701, RFC Editor, jan 2020. URL <https://www.rfc-editor.org/rfc/rfc8701>.
- [8] Michael Larabel . Openssl 4.0 alpha 1 released with encrypted client hello "ech" other features, 2026. URL <https://www.phoronix.com/news/OpenSSL-4.0-Alpha-1>. Accessed: 2026-04-06.
- [9] Stephen Farrell. Defo: Developing encrypted clienthello for openssl, 2024. URL <https://github.com/defo-project/openssl>. Experimental ECH forks for Apache and OpenSSL 3.x.
- [10] Roman Arutyunyan. Encrypted client hello comes to nginx, 2026. URL <https://blog.nginx.org/blog/encrypted-client-hello-comes-to-nginx>. Accessed: 2026-04-06.
- [11] Chrome Platform Status. TLS Encrypted ClientHello. Chromium Dashboard, 2023. URL <https://chromestatus.com/feature/6196703843581952>.
- [12] Google LLC. Boringssl: Bogo tls test suite, 2024. URL <https://boringssl.googlesource.com/boringssl/+HEAD/ssl/test/>.
- [13] Fabian Baumer et al. Tls-attacker: A dynamic framework for analyzing tls implementations. 2024. URL https://www.acsac.org/2024/program/artifacts_competition/comp-acsa24-final11.pdf.
- [14] Dirk Wetter. Testing tls/ssl encryption, 2025. URL <https://testssl.sh/>.
- [15] Alban Diquet. Sslyze: Fast and powerful ssl/tls scanning library, 2025. URL <https://github.com/nabla-c0d3/sslyze>.
- [16] The Apache Software Foundation. Apache httpd test framework (perl-based httpd-test), 2024. URL <https://github.com/apache/httpd/tree/trunk/test>.

- [17] Python Cryptographic Authority. cryptography: Python library which exposes cryptographic recipes and primitives, 2025. URL <https://cryptography.io/en/latest/>.
- [18] The Apache Software Foundation. *Apache HTTP Server Version 2.4 Documentation: mod_ssl Environment Variables*, 2024. URL https://httpd.apache.org/docs/2.4/mod/mod_ssl.html#envvars.

Appendix A

First Appendix

This is the first appendix. For more appendices, add `\chapter{appendix title}` for another appendix.